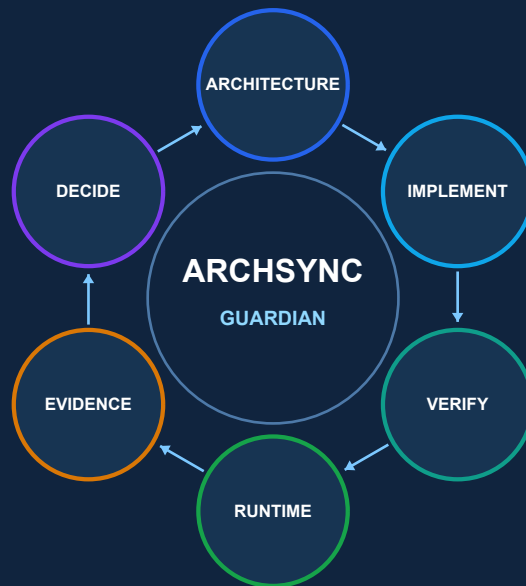


ARCHSYNC

Lộ trình nghiên cứu và phát triển
Closed-Loop Living Architecture Engineering



DÍCH MVP

Architecture Model <-> Source Code
phát hiện drift bằng deterministic core

TẦM NHÌN

Design <-> Code <-> IaC <-> Runtime
đánh giá tiến hóa kiến trúc bằng bằng chứng

PHẦN 01

Tóm tắt điều hành

Một cách hiểu thống nhất để cả nhóm cùng bắt đầu.

Ý tưởng trong một câu ArchSync biến software architecture từ tài liệu tĩnh thành một model có thể đọc bằng máy, liên tục so sánh với code, hạ tầng và runtime; Guardian dùng bằng chứng để phân biệt vi phạm, thay đổi không ảnh hưởng và tiến hóa kiến trúc hợp lệ.	
Vấn đề AI coding agent có thể tối ưu cục bộ nhưng vô tình tạo dependency, trust boundary hoặc topology trái với thiết kế toàn hệ thống.	Artifact cần xây Architecture Guardian gồm Conformance Engine, Evolution Engine, analyzers, CLI/CI và các view sinh từ architecture model.
Cách chứng minh So sánh Expected Graph với Observed Graph trên benchmark có drift được cài có chủ đích; đo precision, recall, repair success và chi phí.	Kết quả mong muốn Một closed loop: Detect -> Explain -> Verify -> Approve -> Synchronize, với quyền tự động hóa tăng dần theo mức rủi ro.

Hai đường đích

Đường đích	Phạm vi	Thời lượng tham chiếu	Điều kiện coi là thành công
MVP kỹ thuật	Architecture Model <-> Source Code	12 tuần	CLI phát hiện đúng drift trên benchmark, có file/line evidence, chạy được trên PR.
Research prototype	Design <-> Code <-> IaC <-> Runtime	28 tuần	Đánh giá được conformance và evolution bằng evidence, có benchmark + báo cáo thực nghiệm tái lập.

Bốn quyết định khóa

- **architecture.yaml là source of truth**; Draw.io, Mermaid và Web UI chỉ là view được sinh lại từ model.
- **Deterministic engine là nền móng**; LLM giải thích và đề xuất, không làm judge cuối cùng cho hard constraints.
- **Không tự cập nhật architecture chỉ vì không phạm rule**; topology change phải qua intent, evidence và approval gate.
- Khởi động bằng 5 repository; core sở hữu schema, các repo còn lại giữ boundary kỹ thuật độc lập.

PHẦN 02

Đóng khung nghiên cứu

Tầm nhìn rộng, tuyên bố hẹp, câu hỏi đo được.

Tuyên bố novelty tạm thời

Không tuyên bố phát minh architecture drift detection. Khoảng đóng góp cần kiểm chứng là drift do AI coding agent và cơ chế đồng bộ 4 nguồn bằng runtime evidence. Trước khi viết paper phải hoàn thành systematic literature review trên IEEE, ACM và Scopus.

Research questions đề xuất

RQ	Câu hỏi	Tín hiệu trả lời
RQ1	AI-assisted development tạo architecture drift ở mức nào so với human development?	Violation rate theo commit/feature, loại drift, thời điểm xuất hiện.
RQ2	Kết hợp architecture spec, code, IaC và runtime evidence có cải thiện drift detection không?	Precision, recall, F1, false positive, unsupported claim.
RQ3	LLM có thể giải thích root cause và sinh repair bảo toàn architecture đến đâu?	Repair success, regression rate, verify pass, time-to-fix.
RQ4	Continuous governance giảm drift mà không làm mất quá nhiều productivity không?	Development time, merge delay, token/compute cost, approval load.

Giả thuyết và đóng góp

H1 - Drift AI agent tạo nhiều drift topology hơn khi không được cấp architecture context và guardrails.	H2 - Evidence Multi-source evidence giảm false positive và unsupported claim so với static analysis hoặc LLM-only.
Contribution A Measurement study về architecture drift trong chuỗi feature do human/AI triển khai.	Contribution B Framework và benchmark tái lập cho conformance + evidence-driven evolution.

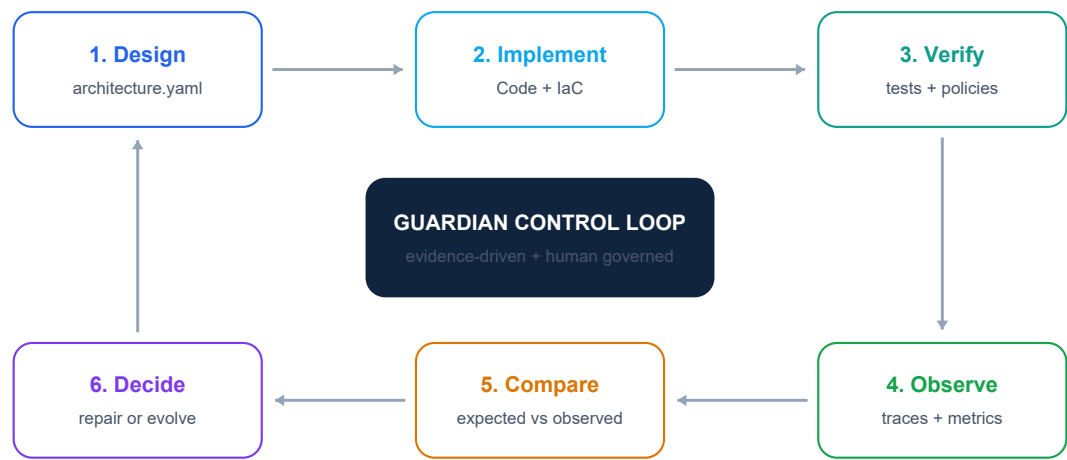
Phạm vi tuyên bố

Được phép nói	Chưa được phép nói
Framework duy trì consistency trên benchmark đã định nghĩa.	ArchSync giải quyết mọi bài toán software architecture.
Runtime evidence cải thiện kết quả theo metric thí nghiệm.	Ý tưởng hoàn toàn chưa từng được công bố.
LLM sinh candidate repair và explanation có dẫn chứng.	LLM tự quyết architecture nào là tốt nhất.

PHẦN 03

Mô hình vận hành closed loop

Architecture thay đổi có kiểm soát, không đồng bộ mù quáng.



Ba loop chạy theo trigger khác nhau

Loop	Trigger	Guardian làm gì	Ví dụ
Development	Git diff / pull request	Dựng delta graph, kiểm tra rule, báo architecture impact.	AI thêm Backend -> Redis.
Architecture	Architect sửa model/design	Tính impact lên code, IaC, config và test cần đổi.	Thêm API Gateway.
Runtime	Trace/metric/topology đổi	Đối chiếu observed runtime với baseline và code/IaC.	Service gọi DB ngoài đường dự kiến.

Nguyên tắc source of truth

Model có version trong Git là baseline được phê duyệt. Diagram được regenerate từ model. Runtime chỉ phản ánh reality đang quan sát, không tự động trở thành design hợp lệ.

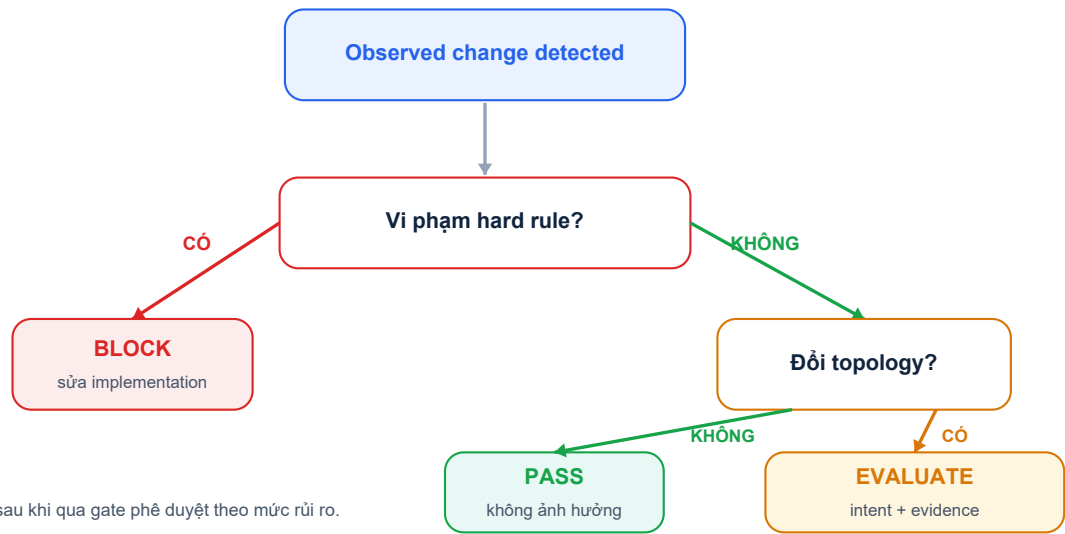
Một vòng hoàn chỉnh phải tạo được evidence chain

Bước	Câu hỏi kiểm soát	Artifact lưu lại
Propose	Thay đổi muốn đạt requirement nào?	Change proposal + owner + hypothesis.
Implement	Code/IaC nào thay đổi topology?	Git diff + observed graph delta.
Measure	Kết quả thật có tốt hơn baseline?	Test, load test, traces, cost, security evidence.
Evaluate	Benefit có đáng với trade-off?	Scorecard + rule result + recommendation.
Accept/Reject	Ai có thẩm quyền quyết định?	Approval record hoặc repair request.
Synchronize	Baseline và views đã nhất quán?	architecture.yaml + generated diagram + changelog.

PHẦN 04

Logic quyết định và governance

Rule, topology và chất lượng là ba câu hỏi khác nhau.



Chỉ cập nhật baseline sau khi qua gate phê duyệt theo mức rủi ro.

Ba trạng thái đầu ra bắt buộc

Trạng thái	Ví dụ	Xử lý mặc định
VIOLATION	Frontend -> Database bị policy DENY.	Block merge hoặc tạo repair task. Không sửa architecture để hợp thức hóa code.
NO IMPACT	Đổi thuật toán bên trong service, topology giữ nguyên.	Pass; lưu evidence ở mức tối thiểu.
VALID EVOLUTION	Order Service thêm Redis, không phạm rule nhưng đổi topology.	Đánh giá intent + quality goals + evidence; approve rồi mới update baseline.

Approval matrix

Mức rủi ro	Dấu hiệu	Quyền quyết định	Tự động hóa
LOW	Internal dependency đã nằm trong allowlist; không đổi data/trust boundary.	Policy owner hoặc auto-policy.	Auto verify; có thể auto-sync sau pilot.
MEDIUM	Thêm cache/service; tăng cost/ops; thay API contract nội bộ.	Tech lead + architect.	Guardian recommendation; human approval.
HIGH	Public exposure, database mới, trust boundary, split service, message broker.	Architect bắt buộc; security/SRE khi liên quan.	Không auto-accept; đầy đủ evidence và rollback.

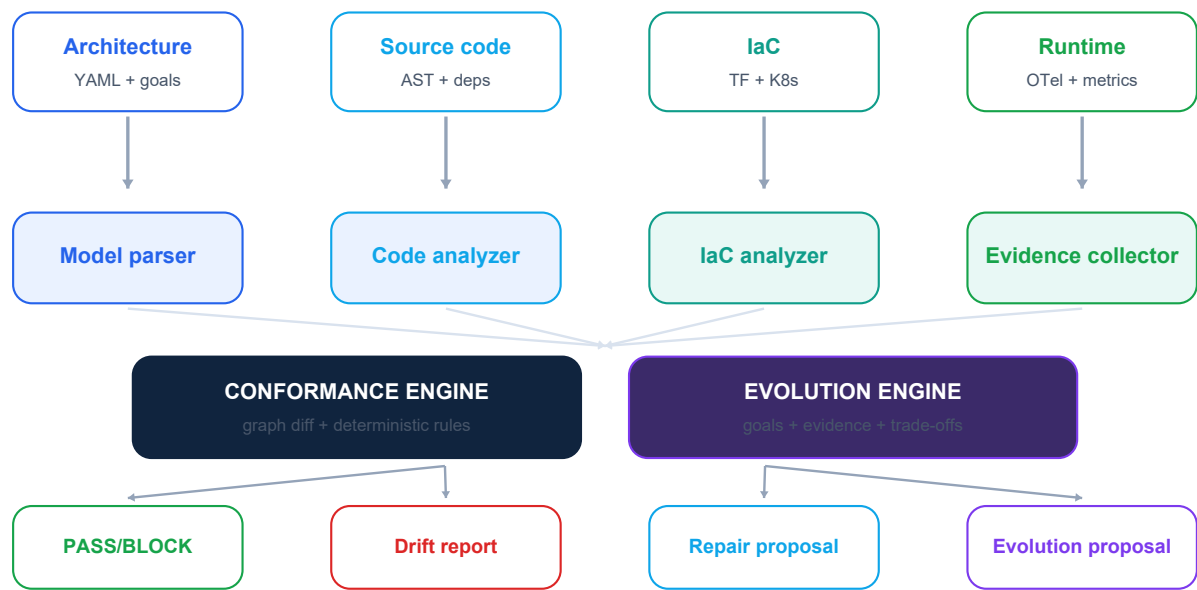
Điều kiện cập nhật architecture.yaml

Change intentional + không vi phạm hard constraint + đạt quality target quan trọng + trade-off trong ngưỡng + có owner/rollback + approval đúng cấp. Thiếu một điều kiện thì baseline chưa đổi.

PHẦN 05

Kiến trúc giải pháp mục tiêu

Hai engine, nhiều evidence source, một audit trail.



Các khối chính

Khối	Input	Output	Nguyên tắc
Architecture Model	YAML schema, relationships, rules, quality goals	Expected Graph + policy set	Versioned, reviewable, machine-readable.
Code Analyzer	AST, imports, clients, routes, manifests	Observed Code Graph + evidence location	Ngôn ngữ nào hỗ trợ phải có precision benchmark.
IaC Analyzer	Terraform, Docker, Kubernetes	Observed Infra Graph	Không đưa vào MVP 12 tuần.
Runtime Collector	OpenTelemetry traces, metrics, logs	Observed Runtime Graph	Evidence có timestamp và confidence.
Conformance Engine	Expected + observed graphs + rules	Pass, drift, violation	Deterministic trước; giải thích được.
Evolution Engine	Goals, runtime, cost, security, candidate	Trade-off scorecard + recommendation	LLM hỗ trợ reasoning; human gate.
Views/Integrations	Approved model + findings	CLI, CI check, Mermaid, Draw.io, Web UI, MCP	View không sở hữu business logic.

Evidence record tối thiểu

Trường	Ý nghĩa	Ví dụ
claim	Kết luận máy đưa ra	New dependency: frontend -> payment-service
evidence	Vị trí hoặc phép đo hỗ trợ	frontend/src/api/payment.ts:24
rule / goal	Contract được kiểm tra	ARCH-002 / p95 < 200 ms
confidence	Độ chắc của inference	0.91; bắt buộc với heuristic/LLM
decision	Kết quả và người phê duyệt	BLOCK / architect@team

PHẦN 06

Phạm vi MVP 12 tuần

Chứng minh core trước khi thêm AI, MCP, IaC hoặc runtime.

IN - bắt buộc	OUT - để sau MVP
Schema architecture.yaml: components, relationships, rules.	Tự động sửa code hoặc tự cập nhật architecture.
Parser + graph builder + graph diff + rule engine.	Terraform/Kubernetes/OpenTelemetry analyzers.
Một Code Analyzer cho đúng một stack benchmark.	Dashboard hoàn chỉnh hoặc Draw.io editor hai chiều.
CLI check repo và check --diff main.	MCP server như interface chính.
Evidence: file/line, node/edge, rule id, severity.	LLM làm judge hoặc scoring chính.
CI gate và bộ test drift có ground truth.	Hỗ trợ đa ngôn ngữ quá sớm.

Milestone v0.1

Expected Graph khác Observed Graph phải tạo được một finding có thể tái lập

Ví dụ: frontend -> payment-service, rule ARCH-002, source frontend/src/api/payment.ts:24, status BLOCK. Cùng input phải cho cùng output; không cần LLM để đạt milestone này.

Benchmark lab tối thiểu

5 components frontend, gateway, order-service, payment-service, database.	10 changes 5 hợp lệ, 3 violation, 2 valid evolution để kiểm tra classification.
Ground truth Mỗi case có expected nodes/edges, rule result, evidence location.	1 stack Chọn một ngôn ngữ/framework mà team hiểu sâu để đo precision thực sự.

Cấu trúc 5 repository hiện tại

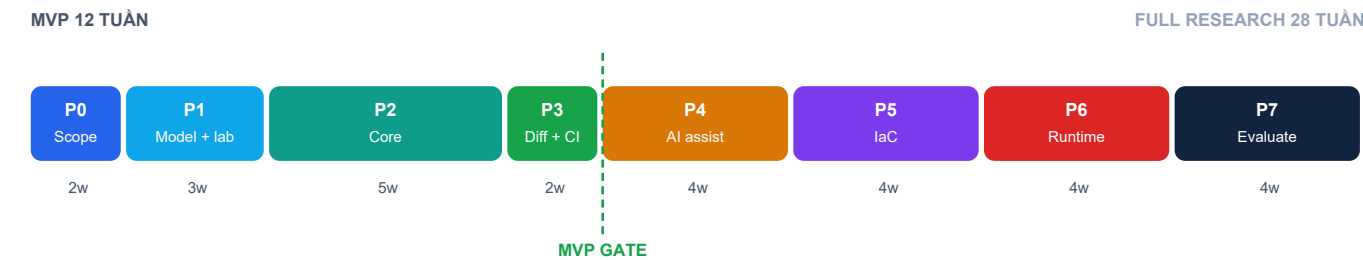
Repositories:

archsnc-core	# model, schema, graph
archsnc-guardian	# conformance control plane
archsnc-mcp	# MCP adapter boundary
archsnc-benchmark	# ground truth + patches
archsnc-examples	# models, views, roadmap

PHẦN 07

Roadmap tổng thể 28 tuần

MVP ở tuần 12; research release ở tuần 28.



P	Tuần	Giai đoạn	Kết quả chính	Gate / phiên bản
0	1-2	Scope + literature	RQ, protocol, terminology, decision log	Research baseline
1	3-5	Model + benchmark lab	Schema, graph model, 10 ground-truth cases	v0.0
2	6-10	Deterministic Guardian Core	Parser, analyzer, diff, rules, CLI	v0.1 core
3	11-12	Git diff + CI	PR finding, regression suite, reproducible report	MVP / v0.2
4	13-16	AI assistance	Explanation, root cause, repair candidates, verification	v0.3
5	17-20	laC synchronization	Terraform/K8s graph + cross-source conflict	v0.4
6	21-24	Runtime + Evolution Engine	OTel evidence, goals, trade-off scorecard	v0.5
7	25-28	Experiment + paper	Dataset, ablation, analysis, artifact package	v1.0 research

Chuỗi phụ thuộc không được đảo

Phải có trước	Mới được làm sau	Lý do
Ground truth benchmark	Tối ưu analyzer	Không có truth thì không đo được precision/recall.
Deterministic findings	LLM explanation	LLM phải giải thích evidence có thật, không tự bịa lỗi.
Code graph ổn định	laC/runtime fusion	Tránh debug nhiều nguồn cùng lúc.
Quality goals	Evolution recommendation	Không có mục tiêu thì không thể nói architecture tốt hơn.
Verification sandbox	Repair generation	Candidate không được chấp nhận trước khi kiểm chứng.

Quy tắc stop/go

Nếu v0.1 không đạt precision/recall mục tiêu trên benchmark, không mở rộng sang AI/laC/runtime. Dành iteration tiếp theo cho analyzer và data quality.

PHẦN 08

Chi tiết giai đoạn 0-1

Khóa câu hỏi nghiên cứu, rồi xây model và lab.

GIAI ĐOẠN 0				Tuần 1-2
Scope, literature và research protocol				Research baseline
Mục tiêu	Chuyển tầm nhìn thành câu hỏi đo được và tạo cơ chế kiểm chứng novelty.			
Công việc	- Lập search protocol + inclusion/exclusion criteria. - Tạo literature matrix và glossary thống nhất. - Chốt RQ, hypotheses, metrics, ethical/reproducibility notes. - Mở ADR cho các quyết định phạm vi.	Đầu ra	- RESEARCH.md - literature-matrix.csv - experiment-protocol.md - ADR-0001 scope	
Exit gate	- Mọi RQ có metric và data source. - Không còn dùng lẫn architecture, model, diagram. - Novelty được viết ở dạng claim cần kiểm chứng.	Owner / phụ thuộc	Research lead + architect Phụ thuộc: quyền truy cập cơ sở dữ liệu học thuật.	

GIAI ĐOẠN 1				Tuần 3-5
Architecture model và benchmark lab				v0.0
Mục tiêu	Biến architecture thành contract đọc được bằng máy và tạo ground truth để phát triển analyzer.			
Công việc	- Thiết kế schema v0.1: components, edges, rules, goals. - Tạo graph domain model + validation. - Xây microservice benchmark một stack. - Cài 10 change cases và expected findings.	Đầu ra	- architecture.schema.json - architecture.yaml mẫu - benchmark repo - ground-truth.json - schema tests	
Exit gate	- Schema reject input sai có message rõ. - 10/10 case có owner và expected graph delta. - Diagram Mermaid sinh được từ model.	Owner / phụ thuộc	Core engineer + research engineer Phụ thuộc: taxonomy từ phase 0.	

PHẦN 09

Chi tiết giai đoạn 2-3

Tạo Guardian Core và đưa vào pull request.

GIAI ĐOẠN 2			Tuần 6-10
Deterministic Guardian Core			v0.1
Mục tiêu	Từ architecture.yaml và repository tạo expected/observed graphs, rồi phân loại pass, violation, evolution.		
Công việc	- Parser + graph builder + graph diff. - Code analyzer cho một stack. - Rule engine deny/allow/required path. - Finding schema có file/line evidence. - CLI archsync check.	Đầu ra	- core library - analyzer plugin v1 - CLI - finding JSON + human report - unit/integration tests
Exit gate	- Precision và recall ≥ 0.85 trên lab. - Không có nondeterministic output. - Mỗi violation map về rule + evidence. - Valid evolution không bị đánh đồng với violation.	Owner / phụ thuộc	Core/analyzer engineers Phụ thuộc: schema + ground truth ổn định.

GIAI ĐOẠN 3			Tuần 11-12
Git diff, CI gate và MVP			v0.2 / MVP
Mục tiêu	Phân tích tác động của thay đổi mới thay vì chỉ scan toàn repo, đồng thời tạo gate có thể dùng trong team.		
Công việc	- Implement check --diff . - PR annotation + exit code theo severity. - Cache graph baseline và incremental update. - Regression suite cho 10 change cases. - Viết quickstart 15 phút.	Đầu ra	- CI workflow - diff report - MVP demo - performance baseline - user guide
Exit gate	- PR vi phạm bị block đúng. - PR no-impact pass. - PR evolution tạo approval request. - Demo tái lập trên máy mới trong ≤ 15 phút.	Owner / phụ thuộc	Platform engineer + core engineer Phụ thuộc: finding schema v0.1.

PHẦN 10

Chi tiết giai đoạn 4-5

Thêm AI có ràng buộc, sau đó hợp nhất code và IaC.

GIAI ĐOẠN 4

Tuần 13-16

v0.3

AI explanation và repair candidates

Mục tiêu	Dùng LLM để giải thích, tìm root cause và sinh candidate repair dựa trên deterministic evidence.		
Công việc	- Prompt contract chỉ nhận evidence có cấu trúc. - Explanation có citation về file/rule/metric. - Repair candidate + impact summary. - Verification sandbox: tests + conformance recheck. - Ablation LLM-only vs evidence-grounded.	Đầu ra	- reasoner service - repair schema - verification runner - ablation report - safety policy
Exit gate	- Unsupported claim rate <= 10%. - Không auto-merge repair. - Mọi candidate phải qua deterministic recheck. - Giảm time-to-explain so với manual baseline.	Owner / phụ thuộc	AI engineer + core engineer Phụ thuộc: stable findings; credential/security review.

GIAI ĐOẠN 5

Tuần 17-20

laC synchronization

v0.4

Mục tiêu	Mở rộng observed architecture sang hạ tầng và phân giải xung đột giữa code với deployment reality.		
Công việc	- Terraform/Kubernetes parsers có scope hẹp. - Infra graph normalization. - Cross-source evidence linking. - Conflict classification: spec-code-laC. - Thêm security policies: public DB, trust boundary.	Đầu ra	- laC analyzer - normalized graph - conflict report - laC benchmark cases - security rule pack
Exit gate	- Phát hiện đủ case public exposure trong lab. - Mỗi conflict nêu nguồn nào nói gì. - Không thay baseline khi evidence mâu thuẫn.	Owner / phụ thuộc	Infra engineer + security reviewer Phụ thuộc: graph identity mapping.

PHẦN 11

Chi tiết giai đoạn 6-7

Runtime evidence, evolution và research release.

GIAI ĐOẠN 6			Tuần 21-24
Runtime evidence và Evolution Engine			v0.5
Mục tiêu	Đánh giá candidate architecture theo requirement và bằng chứng đo được, không theo cảm giác của LLM.		
Công việc	- OpenTelemetry collector + runtime graph. - Quality goals: latency, availability, security, cost, complexity. - Before/after scorecard. - Risk classification + approval workflow. - Regenerate Mermaid/Draw.io view từ approved model.	Đầu ra	- runtime collector - goal schema - trade-off report - approval record - generated views
Exit gate	- Candidate Redis được accept/reject đúng theo goals. - Scorecard có provenance và timestamp. - High-risk change luôn cần human approval. - Baseline/views đồng bộ sau accept.	Owner / phụ thuộc	SRE/telemetry + architect Phụ thuộc: IaC graph và verification sandbox.

GIAI ĐOẠN 7			Tuần 25-28
Thực nghiệm, phân tích và artifact package			v1.0 research
Mục tiêu	Trả lời RQ bằng thí nghiệm tái lập và đóng gói prototype để người ngoài kiểm tra.		
Công việc	- Chạy measurement study theo protocol. - Ablation: code-only, multi-source, LLM-only, evidence-grounded. - Phân tích statistical + qualitative error taxonomy. - Viết paper/report và artifact README. - Freeze dataset, seeds, versions, cost logs.	Đầu ra	- dataset - results.csv - analysis notebook - paper/report - artifact package + release
Exit gate	- Mỗi RQ có bảng/figure + conclusion. - Kết quả chạy lại được. - Threats to validity được ghi rõ. - Tất cả claim truy về data.	Owner / phụ thuộc	Research lead + toàn team Phụ thuộc: freeze v0.5 và protocol phase 0.

PHẦN 12

Thiết kế thực nghiệm

Đo drift, detection, repair và productivity trên cùng một chuỗi feature.

Bốn nhóm so sánh

Nhóm	Điều kiện	Mục đích
A - Human	Developer làm feature với tài liệu architecture.	Baseline thực hành thông thường.
B - AI	Coding agent không có architecture context/guardrail.	Đo drift do tối ưu cục bộ.
C - AI + Context	Agent được cung cấp model/rules nhưng không có Guardian gate.	Tách hiệu ứng của context.
D - AI + ArchSync	Agent + deterministic gate + evidence-grounded repair.	Đo hiệu quả framework.

Quy trình một run

Bước	Kiểm soát	Dữ liệu ghi lại
1. Baseline	Cùng repo, model, test, tool versions.	Initial graph + environment hash.
2. Feature sequence	Cùng task order và acceptance tests.	Commits, prompts, wall time, tokens.
3. Ground-truth review	Hai reviewer độc lập; resolve disagreement.	True nodes/edges/violations.
4. Guardian run	Cố định config và model version.	Findings, confidence, latency, cost.
5. Repair/approve	Verification sandbox bắt buộc.	Repair diff, tests, decision, rollback.
6. Analysis	Ablation + error taxonomy.	Metrics table + qualitative cases.

Metric chính và tiêu chí MVP

Nhóm metric	Metric	Công thức / cách đo	Mục tiêu ban đầu
Detection	Precision / Recall / F1	So finding với ground truth.	≥ 0.85 trên benchmark
Explanation	Unsupported Claim Rate	Claim không có evidence / tổng claim.	$\leq 10\%$ ở phase 4
Repair	Repair Success / Regression	Pass conformance + tests / lỗi mới.	Báo cáo, chưa đặt target sớm
Drift	Violation Rate	Violation / commit hoặc feature.	So sánh A-B-C-D
Productivity	Time, merge delay, approval load	Thời gian + số lần human can thiệp.	Không giảm $> 20\%$ ở pilot
Efficiency	Token, compute, runtime	Chi phí cho mỗi finding/repair đúng.	Ghi đầy đủ provenance

Threats to validity cần ghi ngay từ đầu

Benchmark quá nhỏ, reviewer bias, một stack duy nhất, model drift, prompt sensitivity, data leakage và khác biệt trình độ developer đều có thể làm kết luận sai. Protocol phải khóa trước khi chạy thử nghiệm chính.

PHẦN 13

Tổ chức nhóm và nhịp làm việc

Rõ owner, rõ quyết định, rõ artifact.

Vai trò đề xuất cho team 4-5 người

Vai trò	Trách nhiệm chính	Không được bỏ qua
Research lead / architect	RQ, scope, architecture model, approval policy, paper.	Threats to validity và decision log.
Core / graph engineer	Schema, parser, graph diff, rules, finding model.	Determinism, versioning, test coverage.
Analyzer engineer	Code/laC inference, evidence location, precision tuning.	Ground truth và error taxonomy.
AI / verification engineer	Reasoner, repair candidate, sandbox, ablation.	Không để LLM bypass hard constraints.
SRE / experiment engineer	CI, OTel, benchmark runner, data capture, release.	Reproducibility và cost logs.

RACI tối giản

Artifact / quyết định	Research	Core	Analyzer	AI/SRE	Architect
RQ + protocol	R	C	C	C	A
architecture schema	C	R	C	I	A
ground truth	A	C	R	C	C
hard rule result	I	R	C	I	A
repair proposal	C	C	C	R	A
high-risk evolution	C	I	I	C	A/R
paper claim	R	C	C	C	A

Nhịp tuần

Thứ 2 - Plan Chốt experiment/task, owner, expected evidence và exit gate.	Thứ 4 - Evidence review Xem false positive/negative, thay đổi taxonomy và ADR nếu cần.
Thứ 6 - Demo + decision Demo trên benchmark; accept/reject theo gate, không theo phần trăm hoàn thành.	Cuối tuần - Freeze Lưu tool/model versions, result snapshot và tuần kế tiếp không sửa lịch sử data.

Quy ước repository

- Mọi architecture change đi qua pull request riêng, có rationale, evidence và risk level.
- Mọi finding schema thay đổi phải version và migration note; benchmark giữ được replay.
- Code Analyzer plugin có bảng support matrix và precision report riêng.
- Paper claim chỉ được merge khi có link tới result artifact hoặc literature source.

PHẦN 14

Kế hoạch 10 ngày làm việc đầu tiên

Danh sách có thể đưa thẳng vào sprint đầu.

Ngày	Việc chính	Đầu ra cuối ngày	Owner
1	Kickoff: thống nhất glossary, user/problem, MVP gate, stack benchmark.	ADR-0001 + glossary + decision board	Architect
2	Viết RQ, hypotheses, metrics và danh sách claim cần kiểm chứng.	RESEARCH.md v0.1	Research
3	Thiết kế search protocol và literature matrix.	protocol.md + matrix template	Research
4	Thiết kế architecture schema v0.1 và 3 YAML examples.	schema draft + examples	Core
5	Review schema bằng 10 topology scenarios; khóa identifiers.	schema v0.1 + ADR-0002	Core + Analyzer
6	Scaffold 5 repositories, CI test, package boundaries.	5 repositories có boundary rõ ràng	Platform
7	Build parser + graph domain model + validation tests.	Expected Graph từ YAML	Core
8	Tạo benchmark 5 components và 10 change cases.	benchmark repo + task list	Analyzer
9	Gắn ground truth nodes/edges/rules/evidence cho từng case.	ground-truth.json	Research + Analyzer
10	Demo Graph Diff bằng input giả lập; retro và kế hoạch phase 2.	v0.0 demo + backlog được ưu tiên	Toàn team

Exit gate sprint đầu

MODEL Một architecture.yaml hợp lệ tạo đúng graph; input sai bị reject rõ ràng.	LAB 10 case có ground truth và acceptance criteria, không phụ thuộc LLM.
RESEARCH RQ/metric/protocol đủ để biết dữ liệu nào phải ghi từ bây giờ.	TEAM Mọi artifact có owner; mọi scope change đi qua ADR.

Bốn câu phải trả lời trong kickoff

- Stack benchmark đầu tiên là gì, và ai đủ sâu để review analyzer inference?
- Target user đầu tiên là architect, platform team hay developer trong pull request?
- MVP dùng rule types nào: deny edge, allowed path, layer boundary, required dependency?
- Ngưỡng nào block PR; ngưỡng nào chỉ warning; ai phê duyệt valid evolution?

Việc chưa làm trong 10 ngày đầu
Không xây dashboard, không đăng ký MCP interface, không hỗ trợ đa ngôn ngữ, không tích hợp runtime và không cho LLM sửa architecture. Mục tiêu duy nhất là model + lab + protocol.

PHẦN 15

Rủi ro, định nghĩa hoàn thành và bàn giao

Để nhóm biết khi nào nên tiến, dừng hoặc thu hẹp.

Risk register ưu tiên

Rủi ro	Tín hiệu sớm	Giảm thiểu	Owner
Analyzer suy luận sai	False positive/negative tăng	Một stack, ground truth, error taxonomy, confidence.	Analyzer
Scope phình	Mở IaC/runtime trước MVP	Stop/go gate tuần 12; OUT list bắt buộc.	Architect
LLM hallucination	Claim thiếu evidence	Structured evidence, citations, deterministic recheck.	AI
Baseline bị hợp thức hóa	Code sai kéo architecture theo	Intent + goal + risk approval; audit log.	Architect
Benchmark quá dễ	F1 cao nhưng lỗi thực tế	Holdout cases, real repo pilot, mutation testing.	Research
Không tái lập	Kết quả đổi theo model/tool	Freeze versions, prompts, seeds, cost, environment hash.	SRE
Novelty yếu	Paper gần giống xuất hiện	Literature update theo tháng; thu hẹp claim/contribution.	Research

Definition of Done cho research release

- Prototype chạy từ clean environment và tạo cùng findings/result tables.
- Mỗi RQ có dataset, analysis, conclusion và threats to validity.
- Mọi architecture decision có evidence, risk level, approver và rollback path.
- Diagram/README được sinh từ approved model hoặc chỉ rõ phần nào là thủ công.
- Artifact package không chứa secret, credential, private telemetry hoặc dữ liệu không được phép công bố.

Bước tiếp theo sau khi nhóm đọc tài liệu

Tổ chức kickoff 60 phút, khóa bốn câu ở trang 14, gán owner cho phase 0-1 và chỉ mở backlog phase 2 sau khi sprint đầu qua exit gate.

Nguồn và ghi chú

Tài liệu này được hệ thống hóa từ cuộc trao đổi ý tưởng do người dùng cung cấp: [ChatGPT - Ý tưởng nghiên cứu AI](#). Nội dung về novelty trong cuộc trao đổi được coi là giả thuyết làm việc, không phải kết luận literature review.

TÊN DỰ ÁN	PHIÊN BẢN	TRẠNG THÁI
ArchSync	Roadmap 1.0	Sẵn sàng kickoff